Knowledge Graph Learning Pipeline

LLM-Assisted Graph Building with Human Confirmation

Implementation Plan v1.0

Project: Al-Furqan — Knowledge Graph Population from Scholarly Lessons **Version:** 1.0 **Date:** March 22, 2026 **Source Material:** دروس مدارسة سورة الأنعام — الشيخ أحمد السيد (20 حلقة) **Status:** Ready for Implementation

---

1. Vision

The Problem

The Knowledge Graph currently has 73 nodes + 95 edges — all manually curated. We have 20 episodes of scholarly lessons (~17 hours) that contain thousands of relationships between Quranic verses, hadith, fiqh rules, and concepts.

Extracting these relationships manually would take weeks. Having the LLM extract them automatically risks inaccuracy.

The Solution

LLM proposes, Human confirms, Code commits.

```
Lesson Audio
    ↓
Whisper (transcription)
    ↓
LLM/Qwen (extracts PROPOSED relationships)
    ↓
Human Review Queue (scholar reviews each proposed edge)
    ↓
Confirmed edges → Knowledge Graph
    ↓
Rejected edges → Feedback → LLM learns what NOT to propose
```

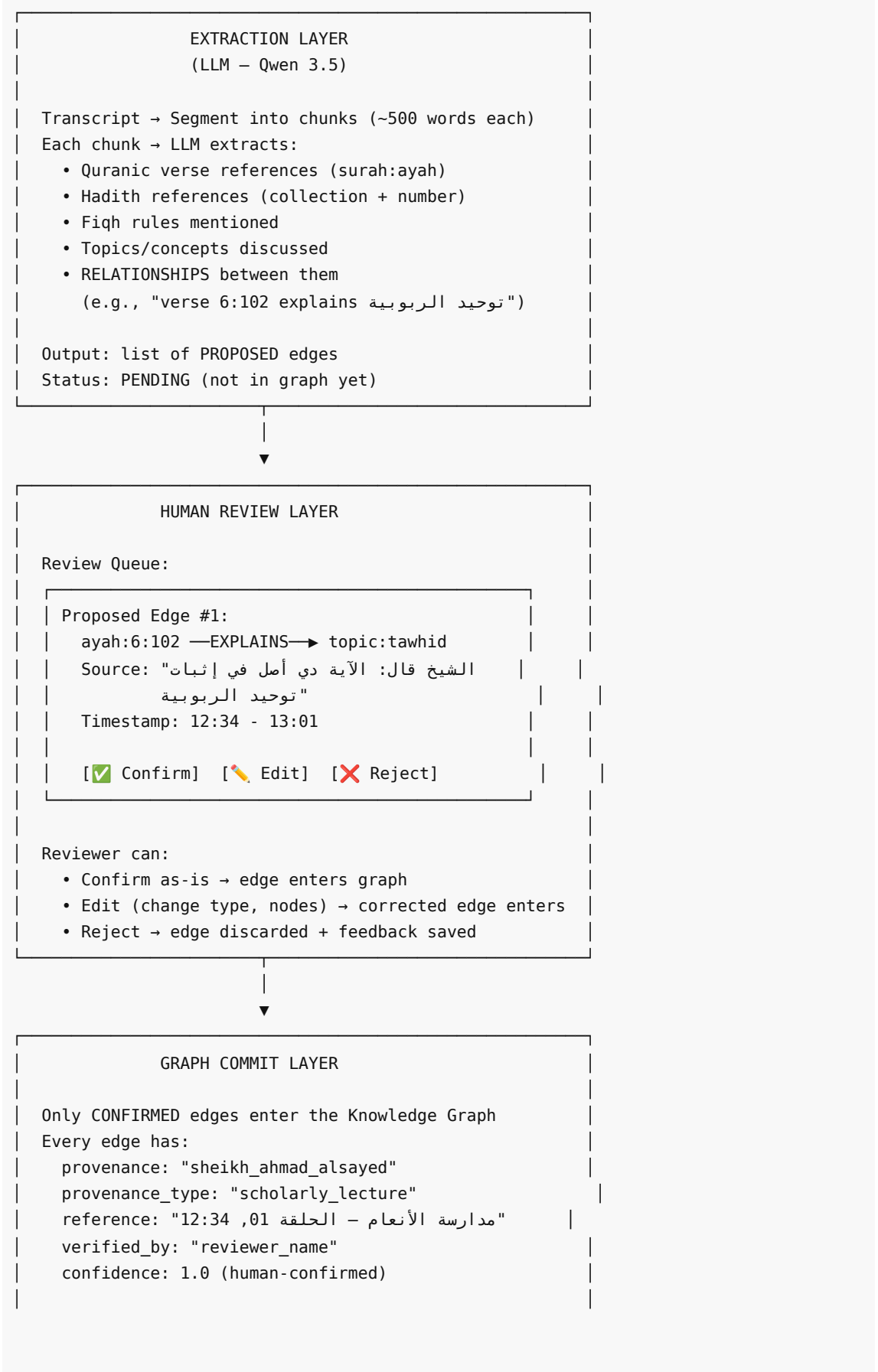
Nothing enters the graph without human confirmation. Ever.

---

2. Architecture

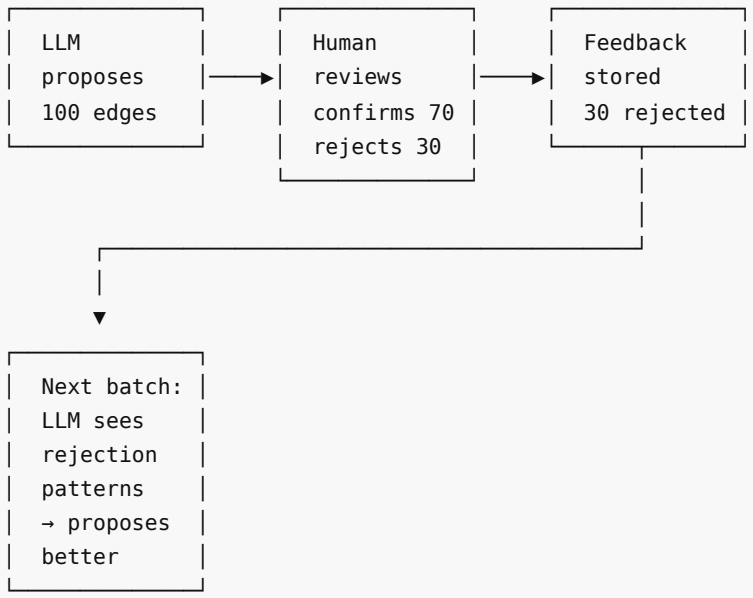
2.1 Pipeline Overview

```
┌─────────────────────────────────────────────────────────┐
│                    INPUT LAYER                          │
│                                                         │
│  YouTube Video → yt-dlp → Audio → Whisper → Transcript  │
│  (already done for Episode 01)                          │
└─────────────────────────┬───────────────────────────────┘
                          │
                          ▼
```


EXTRACTION LAYER
(LLM — Qwen 3.5)
Transcript → Segment into chunks (~500 words each)
Each chunk → LLM extracts:
• Quranic verse references (surah:ayah)
• Hadith references (collection + number)
• Fiqh rules mentioned
• Topics/concepts discussed
• RELATIONSHIPS between them
(e.g., "verse 6:102 explains توحيد الربوبية")
Output: list of PROPOSED edges
Status: PENDING (not in graph yet)
HUMAN REVIEW LAYER
Review Queue:
Proposed Edge #1:
ayah:6:102 ──EXPLAINS──▶ topic:tawhid
Source: "الشيخ قال: الآية دي أصل في إثبات توحيد الربوبية"
Timestamp: 12:34 - 13:01
[✅ Confirm] [✏️ Edit] [❌ Reject]
Reviewer can:
• Confirm as-is → edge enters graph
• Edit (change type, nodes) → corrected edge enters
• Reject → edge discarded + feedback saved
GRAPH COMMIT LAYER
Only CONFIRMED edges enter the Knowledge Graph
Every edge has:
provenance: "sheikh_ahmad_alsayed"
provenance_type: "scholarly_lecture"
reference: "مدارسة الأنعام — الحلقة 01, 12:34"
verified_by: "reviewer_name"
confidence: 1.0 (human-confirmed)

```
│  GraphStore(enforce_provenance=True)                    │
└─────────────────────────────────────────────────────────┘
```

2.2 Feedback Loop




```
Over time: rejection rate drops from 30% → 10% → 5%
```

---

3. Data Model

3.1 Proposed Edge

```python
@dataclass
class ProposedEdge:
    """An edge proposed by the LLM, pending human review."""
    id: str                     # UUID
    source_node: str            # e.g., "ayah:6:102"
    target_node: str            # e.g., "topic:tawhid_rububiyyah"
    edge_type: str              # e.g., "EXPLAINS"

    # Provenance (pre-filled)
    provenance: str             # "sheikh_ahmad_alsayed"
    provenance_type: str        # "scholarly_lecture"
    reference: str              # "مدارسة الأنعام – الحلقة 01"
    timestamp_start: str        # "12:34"
    timestamp_end: str          # "13:01"

    # LLM extraction context
    transcript_chunk: str       # The actual text the LLM extracted from
    llm_reasoning: str          # Why the LLM thinks this relationship exists
    llm_confidence: float       # LLM's self-assessed confidence (0-1)
```

```
    # Review status
    status: str = "pending"     # pending / confirmed / edited / rejected
    reviewed_by: str = ""       # Who reviewed it
    review_notes: str = ""      # Reviewer's notes
    review_timestamp: float = 0.0

    # If edited
    edited_source: str = ""     # Corrected source node (if changed)
    edited_target: str = ""     # Corrected target node
    edited_type: str = ""       # Corrected edge type
```

3.2 Review Queue Storage

```
CREATE TABLE proposed_edges (
    id TEXT PRIMARY KEY,
    lesson_id TEXT NOT NULL,           -- "anam_ep01"
    source_node TEXT NOT NULL,
    target_node TEXT NOT NULL,
    edge_type TEXT NOT NULL,
    provenance TEXT NOT NULL,
    provenance_type TEXT NOT NULL,
    reference TEXT NOT NULL,
    timestamp_start TEXT,
    timestamp_end TEXT,
    transcript_chunk TEXT,
    llm_reasoning TEXT,
    llm_confidence REAL,
    status TEXT DEFAULT 'pending',     -- pending/confirmed/edited/rejected
    reviewed_by TEXT DEFAULT '',
    review_notes TEXT DEFAULT '',
    review_timestamp REAL DEFAULT 0,
    edited_source TEXT DEFAULT '',
    edited_target TEXT DEFAULT '',
    edited_type TEXT DEFAULT '',
    created_at REAL NOT NULL
);

CREATE INDEX idx_status ON proposed_edges(status);
CREATE INDEX idx_lesson ON proposed_edges(lesson_id);
```

---

4. Implementation Tasks

Sprint KG1 — Extraction Pipeline (Week 1)

KG1.1 — Transcript Chunker

File: `src/al_furqan/kb/ingestion/transcript_chunker.py`

```
class TranscriptChunker:
    """Split a Whisper transcript into reviewable chunks."""
```

```
    def __init__(self, chunk_size: int = 500, overlap: int = 50):
        self.chunk_size = chunk_size  # words per chunk
        self.overlap = overlap

    def chunk(self, transcript: dict) -> list[dict]:
        """
        Input: Whisper transcript JSON
        Output: list of chunks with:
          - text: the chunk text
          - start_time: timestamp of first segment
          - end_time: timestamp of last segment
          - segment_ids: which segments are included
        """
        ...
```

KG1.2 — Relationship Extractor (LLM-based)

File: `src/al_furqan/kb/ingestion/relationship_extractor.py`

```
class RelationshipExtractor:
    """Uses LLM to extract PROPOSED relationships from transcript chunks."""

    def __init__(self, llm_fn, quran_collection=None):
        self.llm_fn = llm_fn
        self.quran = quran_collection  # For verse validation

    def extract(self, chunk: dict, lesson_metadata: dict) -> list[ProposedEdge]:
        """
        Send chunk to LLM with extraction prompt.
        LLM returns structured relationships.
        Each becomes a ProposedEdge with status='pending'.
        """
        prompt = self._build_extraction_prompt(chunk, lesson_metadata)
        response = self.llm_fn(prompt)
        raw_edges = self._parse_response(response)

        # Validate: check verse references exist in Quran DB
        validated = self._validate_references(raw_edges)

        return [
            ProposedEdge(
                id=generate_id(),
                source_node=edge["source"],
                target_node=edge["target"],
                edge_type=edge["type"],
                provenance=lesson_metadata["scholar"],
                provenance_type="scholarly_lecture",
                reference=f"{lesson_metadata['series']} —
{lesson_metadata['episode']}",
                timestamp_start=chunk["start_time"],
                timestamp_end=chunk["end_time"],
```

```
                transcript_chunk=chunk["text"],
                llm_reasoning=edge["reasoning"],
                llm_confidence=edge["confidence"],
                status="pending",
            )
            for edge in validated
        ]

    def _build_extraction_prompt(self, chunk, metadata) -> str:
        """
        Prompt instructs LLM to:
        1. Identify Quranic verse references (surah:ayah format)
        2. Identify hadith references
        3. Identify topics/concepts
        4. Identify RELATIONSHIPS between them
        5. Provide reasoning for each relationship
        6. Self-assess confidence

        Respond in structured JSON.
        """
        return (
            "You are a scholarly assistant analyzing an Islamic lesson
transcript.\n\n"
            f"Scholar: {metadata['scholar']}\n"
            f"Series: {metadata['series']}\n"
            f"Episode: {metadata['episode']}\n\n"
            f"Transcript chunk:\n\"{chunk['text']}\"\n\n"
            "Extract ALL relationships between Quranic verses, hadith, "
            "topics, and concepts mentioned in this chunk.\n\n"
            "For each relationship, provide:\n"
            "- source: the source node (e.g., 'ayah:6:102')\n"
            "- target: the target node (e.g., 'topic:tawhid')\n"
            "- type: relationship type (EXPLAINS, ESTABLISHES, REFERENCES, etc.)\n"
            "- reasoning: WHY this relationship exists (what the sheikh said)\n"
            "- confidence: your confidence 0.0-1.0\n\n"
            "IMPORTANT: Only extract relationships the SCHOLAR explicitly states.\n"
            "Do NOT infer relationships the scholar didn't mention.\n"
            "If unsure, set confidence < 0.5.\n\n"
            "Respond in JSON array format."
        )
```

KG1.3 — Reference Validator

File: `src/al_furqan/kb/ingestion/reference_validator.py`

```
class ReferenceValidator:
    """Validates extracted references against known sources."""

    def __init__(self, quran_collection, hadith_collection=None):
        self.quran = quran_collection
        self.hadith = hadith_collection
```

```
    def validate_verse(self, surah: int, ayah: int) -> bool:
        """Check if verse exists in Quran DB."""
        verse = self.quran.get_verse(surah, ayah)
        return verse is not None

    def validate_hadith(self, collection: str, number: int) -> bool:
        """Check if hadith exists in DB."""
        if self.hadith:
            h = self.hadith.get_hadith(collection, number)
            return h is not None
        return True  # Skip validation if no hadith DB

    def validate_edge(self, edge: dict) -> dict:
        """Validate and annotate an edge with validation status."""
        edge["source_valid"] = self._validate_node(edge["source"])
        edge["target_valid"] = self._validate_node(edge["target"])
        edge["valid"] = edge["source_valid"] and edge["target_valid"]
        return edge
```

KG1.4 — Proposed Edge Store

File: `src/al_furqan/kb/ingestion/proposed_edge_store.py`

```
class ProposedEdgeStore:
    """SQLite storage for proposed edges pending human review."""

    def __init__(self, db_path: str = "data/review/proposed_edges.db"):
        self.db = sqlite3.connect(db_path)
        self._create_tables()

    def save(self, edge: ProposedEdge) -> str: ...
    def get_pending(self, lesson_id: str = None, limit: int = 50) -> list: ...
    def confirm(self, edge_id: str, reviewer: str, notes: str = "") -> bool: ...
    def reject(self, edge_id: str, reviewer: str, notes: str = "") -> bool: ...
    def edit_and_confirm(self, edge_id: str, reviewer: str,
                         edits: dict, notes: str = "") -> bool: ...
    def get_stats(self) -> dict: ...
    def get_rejection_patterns(self) -> list: ...
```

Sprint KG2 — Human Review Interface (Week 2)

KG2.1 — CLI Review Tool

File: `src/al_furqan/kb/ingestion/review_cli.py`

```
class ReviewCLI:
    """Interactive CLI for reviewing proposed edges."""

    def __init__(self, store: ProposedEdgeStore, graph: GraphStore):
        self.store = store
        self.graph = graph
```

```python
    def review_session(self, lesson_id: str = None):
        """Start an interactive review session."""
        pending = self.store.get_pending(lesson_id)
        print(f"\n📋 {len(pending)} edges pending review\n")

        for edge in pending:
            self._display_edge(edge)
            action = input("\n[C]onfirm  [E]dit  [R]eject  [S]kip  [Q]uit: ")

            if action.lower() == 'c':
                self.store.confirm(edge['id'], reviewer=self.reviewer)
                self._commit_to_graph(edge)
                print("✅ Confirmed and added to graph")
            elif action.lower() == 'e':
                edited = self._edit_edge(edge)
                self.store.edit_and_confirm(edge['id'], self.reviewer, edited)
                self._commit_to_graph(edited)
                print("✏️ Edited and added to graph")
            elif action.lower() == 'r':
                reason = input("Rejection reason: ")
                self.store.reject(edge['id'], self.reviewer, reason)
                print("❌ Rejected")
            elif action.lower() == 'q':
                break

    def _commit_to_graph(self, edge):
        """Add confirmed edge to the Knowledge Graph."""
        self.graph.add_edge(
            source=edge['source_node'],
            target=edge['target_node'],
            edge_type=edge['edge_type'],
            provenance=edge['provenance'],
            provenance_type=edge['provenance_type'],
            reference=edge['reference'],
            verified_by=edge['reviewed_by'],
            confidence=1.0,  # Human-confirmed = full confidence
        )
```

KG2.2 — Telegram Review Bot (Future)

```python
# Future: Review via Telegram inline buttons
# Reviewer gets a message with proposed edge
# Taps ✅ Confirm / ✏️ Edit / ❌ Reject
# Edge gets committed or rejected
```

KG2.3 — Review Dashboard API

File: `src/al_furqan/api/routers/review_edges.py`

```python
@router.get("/review/pending")
async def get_pending(lesson_id: str = None, limit: int = 50):
    """Get pending edges for review."""
    ...

@router.post("/review/{edge_id}/confirm")
async def confirm_edge(edge_id: str, reviewer: str, notes: str = ""):
    """Confirm a proposed edge."""
    ...

@router.post("/review/{edge_id}/reject")
async def reject_edge(edge_id: str, reviewer: str, reason: str):
    """Reject a proposed edge."""
    ...
```

Sprint KG3 — Feedback & Learning (Week 3)

KG3.1 — Rejection Pattern Analyzer

File: `src/al_furqan/kb/ingestion/feedback_analyzer.py`

```python
class FeedbackAnalyzer:
    """Analyzes rejection patterns to improve future extraction."""

    def analyze(self, store: ProposedEdgeStore) -> dict:
        """
        Identify patterns in rejected edges:
        - Common rejection reasons
        - Edge types that get rejected most
        - Confidence ranges that correlate with rejection
        - Specific node types that are problematic
        """
        rejections = store.get_rejected()

        return {
            "total_proposed": store.get_stats()["total"],
            "total_confirmed": store.get_stats()["confirmed"],
            "total_rejected": store.get_stats()["rejected"],
            "confirmation_rate": ...,
            "rejection_reasons": ...,    # Categorized
            "worst_edge_types": ...,     # Types with lowest confirmation
            "confidence_vs_confirmation": ...,  # Correlation
            "recommendations": ...,      # How to improve the prompt
        }
```

KG3.2 — Adaptive Extraction Prompt

```python
class AdaptiveExtractor(RelationshipExtractor):
    """Improves extraction prompt based on rejection feedback."""
```

```
    def _build_extraction_prompt(self, chunk, metadata) -> str:
        # Get rejection patterns
        patterns = self.feedback_analyzer.analyze(self.store)

        # Add to prompt:
        base_prompt = super()._build_extraction_prompt(chunk, metadata)

        feedback_section = (
            "\n\nLEARNED FROM PREVIOUS REVIEWS:\n"
            f"- Confirmation rate: {patterns['confirmation_rate']:.0%}\n"
            f"- Common mistakes to AVOID:\n"
        )
        for reason in patterns["rejection_reasons"][:5]:
            feedback_section += f"  • {reason}\n"

        return base_prompt + feedback_section
```

Sprint KG4 — Full Pipeline & Batch Processing (Week 4)

KG4.1 — Batch Lesson Processor

File: `src/al_furqan/kb/ingestion/batch_processor.py`

```
class BatchLessonProcessor:
    """Process entire lesson series end-to-end."""

    def process_lesson(self, transcript_path: str, metadata: dict) -> dict:
        """
        Full pipeline for one lesson:
        1. Chunk transcript
        2. Extract relationships (LLM)
        3. Validate references
        4. Save to review queue
        5. Return stats
        """
        transcript = load_transcript(transcript_path)
        chunks = self.chunker.chunk(transcript)

        all_proposed = []
        for chunk in chunks:
            proposed = self.extractor.extract(chunk, metadata)
            for edge in proposed:
                self.store.save(edge)
                all_proposed.append(edge)

        return {
            "lesson": metadata["episode"],
            "chunks_processed": len(chunks),
            "edges_proposed": len(all_proposed),
            "avg_confidence": sum(e.llm_confidence for e in all_proposed) /
len(all_proposed),
```

```
            "status": "awaiting_review",
        }

    def process_series(self, transcripts_dir: str, series_metadata: dict):
        """Process all episodes in a series."""
        ...
```

KG4.2 — Whisper Batch Transcription

File: `scripts/transcribe_series.py`

```
"""Download and transcribe all episodes from YouTube playlist."""
# For each episode:
# 1. yt-dlp (audio only)
# 2. Whisper (Arabic, medium/large model)
# 3. Save transcript JSON
# 4. Run through extraction pipeline
```

---

5. Security & Quality Controls

5.1 Rules

Rule	Enforcement
No edge enters graph without human confirmation	ProposedEdgeStore → ReviewCLI → GraphStore
Every edge has provenance	GraphStore(enforce_provenance=True)
LLM cannot modify existing edges	LLM only proposes NEW edges
Verse references validated against Quran DB	ReferenceValidator before proposal
Rejection feedback improves future extraction	FeedbackAnalyzer → AdaptiveExtractor
Reviewer identity recorded	verified_by field on every edge

5.2 Quality Metrics

Metric	Target	How Measured
Confirmation rate	>70% (improving over time)	confirmed / total proposed
False positive rate	<10%	edges confirmed then later disputed
Reference accuracy	>95%	verse/hadith references that exist in DB
Coverage	>80% of relationships in lesson	manual audit of random chunks

---

6. Current Data

Available Now

```
data/lessons/
└── lesson_01_transcript.json    ← 219 KB, 1,524 segments
    Scholar: الشيخ أحمد السيد
    Series: مدارسة سورة الأنعام
    Episode: 01 الحلقة
    Duration: ~53 minutes
    Content: 136 Quranic refs, 45 hadith refs, 89 توحيد mentions
```

Needed

```
Episodes 02-20 (19 remaining)
    → Download from YouTube playlist
    → Transcribe with Whisper
    → ~17 hours of scholarly content
    → Estimated: 2,000-5,000 relationships to extract
```

---

7. Execution Timeline

```
Week 1: KG1 — Extraction Pipeline
  • Chunker + Extractor + Validator + ProposedEdgeStore
  • Process Episode 01 as proof of concept
  • Generate first batch of proposed edges

Week 2: KG2 — Review Interface
  • CLI review tool
  • API endpoints for review
  • First human review session on Episode 01 edges

Week 3: KG3 — Feedback & Learning
  • Rejection pattern analysis
  • Adaptive extraction prompt
  • Process Episodes 02-05 with improved extraction

Week 4: KG4 — Batch Processing
  • Transcribe remaining episodes (06-20)
  • Process all with adaptive extractor
  • Review sessions for all episodes
  • Final Knowledge Graph population
```

---

8. Expected Outcome

```
Before:  73 nodes + 95 edges (manual, sample)
After:   ~500+ nodes + 2,000-5,000 edges (scholar-verified)
```

```
Every edge traceable to:
  "الحلقة – مدارسة الأنعام – السيد أحمد الشيخ – X الدقيقة Y:Z"
  Reviewed by: [reviewer name]
  Confidence: 1.0 (human-confirmed)
```

This pipeline ensures the Knowledge Graph grows with scholarly quality while leveraging LLM speed for extraction. The human is always in the loop — the AI suggests, the scholar confirms.